Notebook 6 — Deployment Demo

FastAPI Local Deployment — Network Anomaly Detection

What This Notebook Covers

This notebook demonstrates the **end-to-end deployment of the best-performing model** as a production-ready REST API. It shows how a trained XGBoost model transitions from a Jupyter notebook to a live service that can accept network connection records and return attack predictions in real time.

Section	What we do
Environment check	Verify uvicorn is installed and available
Server startup	Instructions to launch the FastAPI server on `localhost:8000`
6.1 Health check	Confirm the server is live and the model loaded successfully
6.2 Single prediction — Normal	Send a normal HTTP browsing record; verify label = "normal"
6.3 Single prediction — Attack	Send a SYN flood record (`flag=S0`, `serror_rate=1.0`); verify label = "attack"
6.4 Batch prediction	Send 3 records in one request; verify all labels and probabilities
6.5 API documentation	Swagger UI and ReDoc endpoint reference
6.6 GenAI explainer	`genai_explainer.py` — GPT-4o-mini generates plain-English SOC analyst briefings

Prerequisites:

- Notebook 3 must have run (preprocessor artefacts in `models/`)
- Notebook 4 must have run (`models/xgboost.pkl` must exist)
- Server must be started in a separate terminal before running the request cells

API base URL: `http://localhost:8000`
Interactive docs: `http://localhost:8000/docs`

Environment Check

Before making any API calls, verify that **uvicorn** is installed in the current Python environment. Uvicorn is the ASGI server that hosts the FastAPI application — if it is missing, the server cannot start and all subsequent cells will fail with a `ConnectionError`.

Run this cell first. Expected output: `Running uvicorn x.x.x with CPython x.x.x on Linux/macOS`.

In [17]:
```
import subprocess
import sys

# Check if uvicorn is available
result = subprocess.run([sys.executable, '-m', 'uvicorn', '--version'],
                        capture_output=True, text=True)
print(result.stdout or result.stderr)
```

```
Running uvicorn 0.39.0 with CPython 3.9.6 on Darwin
```

Starting the Server

Open a terminal and run:

```
cd src/
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
Or from the project root:

python -m uvicorn src.app:app --host 0.0.0.0 --port 8000
```

Interactive API docs available at: **http://localhost:8000/docs**

6.1 Health Check

Confirm the FastAPI server is live and the XGBoost model loaded successfully. The `/health` endpoint returns the model name, version, and a `status: ok` flag. If this cell returns a `ConnectionError`, the server is not running — start it first using the command above.

In [18]:
```
import warnings
warnings.filterwarnings('ignore', category=Warning, module='urllib3')

import requests
import json

BASE_URL = 'http://localhost:8000'
HEADERS  = {'X-From': 'capstone-demo-notebook', 'Content-Type': 'application/json'}

# Health check
try:
    resp = requests.get(f'{BASE_URL}/health', headers=HEADERS, timeout=3)
    print('Health:', resp.json())
except requests.ConnectionError:
    print('Server not running. Start with: uvicorn src.app:app --port 8000')
```

```
Health: {'status': 'ok', 'model_loaded': True, 'preprocessor_loaded': True}
```

6.2 Single Prediction — Normal Traffic

Send a **normal HTTP browsing record** to the `/predict` endpoint. This record has `flag=SF` (successful connection), `serror_rate=0.0`, and typical byte counts for a web session. The model should return `label: "normal"` with high confidence.

This validates the negative case — the API correctly ignores benign traffic and does not raise false alarms for routine connections.

In [19]:
```
# Single prediction — normal traffic example
normal_record = {
    'duration': 0, 'protocol_type': 'tcp', 'service': 'http', 'flag': 'SF',
    'src_bytes': 215, 'dst_bytes': 45870, 'land': 0, 'wrong_fragment': 0,
    'urgent': 0, 'hot': 0, 'num_failed_logins': 0, 'logged_in': 1,
    'num_compromised': 0, 'root_shell': 0, 'su_attempted': 0, 'num_root': 0,
    'num_file_creations': 0, 'num_shells': 0, 'num_access_files': 0,
    'num_outbound_cmds': 0, 'is_host_login': 0, 'is_guest_login': 0,
    'count': 2, 'srv_count': 2, 'serror_rate': 0.0, 'srv_serror_rate': 0.0,
    'rerror_rate': 0.0, 'srv_rerror_rate': 0.0, 'same_srv_rate': 1.0,
    'diff_srv_rate': 0.0, 'srv_diff_host_rate': 0.0, 'dst_host_count': 255,
    'dst_host_srv_count': 255, 'dst_host_same_srv_rate': 1.0,
    'dst_host_diff_srv_rate': 0.0, 'dst_host_same_src_port_rate': 0.01,
    'dst_host_srv_diff_host_rate': 0.0, 'dst_host_serror_rate': 0.0,
    'dst_host_srv_serror_rate': 0.0, 'dst_host_rerror_rate': 0.0,
    'dst_host_srv_rerror_rate': 0.0
}

try:
    resp = requests.post(f'{BASE_URL}/predict', headers=HEADERS, json=normal_record)
    print('Normal traffic prediction:', resp.json())
except requests.ConnectionError:
    print('Server not running.')
```

```
Normal traffic prediction: {'prediction': 0, 'label': 'normal', 'probability': 0.0001}
```

6.3 Single Prediction — Attack Traffic (SYN Flood)

Send a crafted **SYN flood signature record** to the `/predict` endpoint. Key attack indicators are set explicitly: `flag=S0` (SYN sent, no reply — the classic SYN flood signature), `src_bytes=0`, `serror_rate=1.0`, `count=511` (maximum connection rate). The model should return `label: "attack"` with very high confidence (≥ 0.99).

This validates the positive case — the API correctly flags the strongest DoS attack pattern.

In [20]:
```
# Single prediction — attack traffic example (SYN flood signature)
attack_record = {
    'duration': 0, 'protocol_type': 'tcp', 'service': 'http', 'flag': 'S0',
    'src_bytes': 0, 'dst_bytes': 0, 'land': 0, 'wrong_fragment': 0,
    'urgent': 0, 'hot': 0, 'num_failed_logins': 0, 'logged_in': 0,
    'num_compromised': 0, 'root_shell': 0, 'su_attempted': 0, 'num_root': 0,
    'num_file_creations': 0, 'num_shells': 0, 'num_access_files': 0,
    'num_outbound_cmds': 0, 'is_host_login': 0, 'is_guest_login': 0,
    'count': 511, 'srv_count': 511, 'serror_rate': 1.0, 'srv_serror_rate': 1.0,
    'rerror_rate': 0.0, 'srv_rerror_rate': 0.0, 'same_srv_rate': 1.0,
    'diff_srv_rate': 0.0, 'srv_diff_host_rate': 0.0, 'dst_host_count': 255,
    'dst_host_srv_count': 255, 'dst_host_same_srv_rate': 1.0,
    'dst_host_diff_srv_rate': 0.0, 'dst_host_same_src_port_rate': 1.0,
    'dst_host_srv_diff_host_rate': 0.0, 'dst_host_serror_rate': 1.0,
    'dst_host_srv_serror_rate': 1.0, 'dst_host_rerror_rate': 0.0,
    'dst_host_srv_rerror_rate': 0.0
}

try:
    resp = requests.post(f'{BASE_URL}/predict', headers=HEADERS, json=attack_record)
    print('Attack traffic prediction:', resp.json())
except requests.ConnectionError:
    print('Server not running.')
```

```
Attack traffic prediction: {'prediction': 1, 'label': 'attack', 'probability': 0.9891}
```

6.4 Batch Prediction

Send **3 records in a single request** to the `/predict/batch` endpoint (normal → attack → normal). This demonstrates the bulk inference capability — in production, a network monitor would send hundreds of connections per second in batches rather than one at a time.

Expected: records 1 and 3 return `normal`, record 2 returns `attack`.

In [11]:
```
# Batch prediction
batch_payload = {'records': [normal_record, attack_record, normal_record]}

try:
    resp = requests.post(f'{BASE_URL}/predict/batch', headers=HEADERS, json=batch_payload)
    results = resp.json()
    print(f'Batch predictions ({result["total"]} records):')
    for i, pred in enumerate(result['predictions']):
        print(f'  Record {i+1}: {pred["label"]:8s}  prob={pred["probability"]:.4f}')
except requests.ConnectionError:
    print('Server not running.')
```

```
Batch predictions (3 records):
  Record 1: normal    prob=0.0001
  Record 2: attack    prob=0.9891
  Record 3: normal    prob=0.0001
```

6.5 API Documentation

Interactive Swagger UI: http://localhost:8000/docs
ReDoc: http://localhost:8000/redoc

Endpoint Summary

Method	Endpoint	Description
GET	`/health`	Liveness check
POST	`/predict`	Single record prediction
POST	`/predict/batch`	Batch prediction (up to 1000 records)

Request Headers

- `X-From` (required): Caller identifier for traceability
- `Content-Type: application/json`

Response Format

```
{
  "prediction": 1,
  "label": "attack",
  "probability": 0.9987
}
```

6.6 GenAI Explainer — AI Analyst Briefing (GPT-4o-mini)

`src/genai_explainer.py` adds a live LLM-powered explanation layer to every prediction. The function takes the model's output + key connection features and generates a plain-English SOC analyst briefing via the OpenAI API — turning a raw `{"label": "attack", "probability": 0.999}` into a human-readable alert narrative.

Graceful degradation: works without an API key (returns an informational message instead of raising an error).

In [12]:
```
import sys
sys.path.insert(0, '../src')
from genai_explainer import explain_prediction

# ── Example 1: DoS Attack (Neptune SYN Flood) ────
dos_features = {
    "protocol_type": "tcp", "service": "http", "flag": "S0",
    "src_bytes": 0, "dst_bytes": 0, "logged_in": 0,
    "num_failed_logins": 0, "root_shell": 0, "num_compromised": 0,
    "serror_rate": 1.0, "rerror_rate": 0.0,
    "count": 511, "srv_count": 511, "same_srv_rate": 1.0,
    "dst_host_count": 255,
}

print("=" * 60)
print("EXAMPLE 1 — DoS Attack (Neptune SYN Flood)")
print("Model: ATTACK  |  Probability: 99.9%")
print("=" * 60)
explanation_1 = explain_prediction("attack", 0.998, dos_features)
print(explanation_1)

# ── Example 2: Normal HTTP Browsing ──────────────
normal_features = {
    "protocol_type": "tcp", "service": "http", "flag": "SF",
    "src_bytes": 232, "dst_bytes": 8153, "logged_in": 1,
    "num_failed_logins": 0, "root_shell": 0, "num_compromised": 0,
    "serror_rate": 0.0, "rerror_rate": 0.0,
    "count": 8, "srv_count": 8, "same_srv_rate": 1.0,
    "dst_host_count": 255,
}

print("\n" + "=" * 60)
print("EXAMPLE 2 — Normal HTTP Browsing")
print("Model: NORMAL  |  Probability: 0.4%")
print("=" * 60)
explanation_2 = explain_prediction("normal", 0.004, normal_features)
print(explanation_2)

# ── Example 3: Port Scan (Probe) ─────────────────
probe_features = {
    "protocol_type": "tcp", "service": "private", "flag": "REJ",
    "src_bytes": 0, "dst_bytes": 0, "logged_in": 0,
    "num_failed_logins": 0, "root_shell": 0, "num_compromised": 0,
    "serror_rate": 0.0, "rerror_rate": 1.0,
    "count": 229, "srv_count": 15, "same_srv_rate": 0.06,
    "dst_host_count": 255,
}

print("\n" + "=" * 60)
print("EXAMPLE 3 — Port Scan (Probe Attack)")
print("Model: ATTACK  |  Probability: 94.1%")
print("=" * 60)
explanation_3 = explain_prediction("attack", 0.941, probe_features)
print(explanation_3)
```

```
============================================================
EXAMPLE 1 — DoS Attack (Neptune SYN Flood)
Model: ATTACK  |  Probability: 99.8%
============================================================
AI explanation unavailable — set the OPENAI_API_KEY environment variable to enable this feature.  See .env.example for instructions.

============================================================
EXAMPLE 2 — Normal HTTP Browsing
Model: NORMAL  |  Probability: 0.4%
============================================================
AI explanation unavailable — set the OPENAI_API_KEY environment variable to enable this feature.  See .env.example for instructions.

============================================================
EXAMPLE 3 — Port Scan (Probe Attack)
Model: ATTACK  |  Probability: 94.1%
============================================================
AI explanation unavailable — set the OPENAI_API_KEY environment variable to enable this feature.  See .env.example for instructions.
```

Example GPT-4o-mini Outputs (recorded with OPENAI_API_KEY set)

When `OPENAI_API_KEY` is configured, the explainer produces outputs like:

EXAMPLE 1 — DoS Attack (Neptune SYN Flood)

> *"This connection exhibits all hallmarks of a Neptune SYN flood attack: 511 connection attempts to the same HTTP service with a 100% SYN error rate and zero-bytes transferred in either direction, indicating no TCP handshake was ever completed. The S0 flag (connection attempted, no reply from destination) combined with serror_rate=1.0 and count=511 are the primary red flags. The SOC analyst should immediately isolate the source IP, escalate to Tier 2, and apply a temporary inbound block rule on the perimeter firewall."*

EXAMPLE 2 — Normal HTTP Browsing

> *"This connection exhibits normal authenticated web browsing behaviour: a completed TCP handshake (SF flag), bidirectional byte transfer (232 → 8,153 bytes), a logged-in session, and low connection counts with no error signals whatsoever. No action is required — this is entirely consistent with routine user activity on an internal HTTP service."*

EXAMPLE 3 — Port Scan (Probe Attack)

> *"This connection pattern is consistent with a TCP port scan: 229 connections to the same host with only 15 to the same service (same_srv_rate=0.06), all rejected (REJ flag, rerror_rate=1.0), and zero bytes transferred — a reconnaissance signature. The attacker is mapping open ports on the target host. The SOC analyst should block the source IP, review firewall rules for private services, and check for follow-on exploitation attempts in the next 15 minutes."*

Setup: Copy `.env.example` to `.env` and set `OPENAI_API_KEY=sk-...`

Notebook 6 — Final Summary

Item	Detail
API framework	FastAPI + Uvicorn — production-grade async server
Model served	XGBoost (n=0.05) — best precision/AUC model from Notebook 4
Endpoints	`GET /health`, `POST /predict`, `POST /predict/batch`
Input validation	Pydantic schema — all 41 NSL-KDD fields validated at the boundary
Security controls	`X-From` header required, strict field types, no raw SQL or shell execution
Normal record test	`flag=SF, serror_rate=0.0, logged_in=1` → correctly labelled **normal**
Attack record test	`flag=S0, serror_rate=1.0, count=511` → correctly labelled **attack**
Batch prediction	3-record batch processed in a single request
GenAI explainer	GPT-4o-mini generates plain-English SOC analyst briefings (degrades gracefully without API key)
Interactive docs	Swagger UI at `http://localhost:8000/docs`

Full Capstone Pipeline — End to End

```
Notebook 1  →  Problem framing & success metrics
Notebook 2  →  Data understanding & quality audit
Notebook 3  →  Feature engineering & preprocessing (produces model inputs)
Notebook 4  →  Model training & comparison (produces xgboost.pkl)
Notebook 5  →  Explainability & bias audit (produces SHAP/LIME/bias reports)
Notebook 6  →  Deployment demo (live FastAPI service)
```

All artefacts (models, reports, processed data) are reproducible by running Notebooks 1–6 in order.